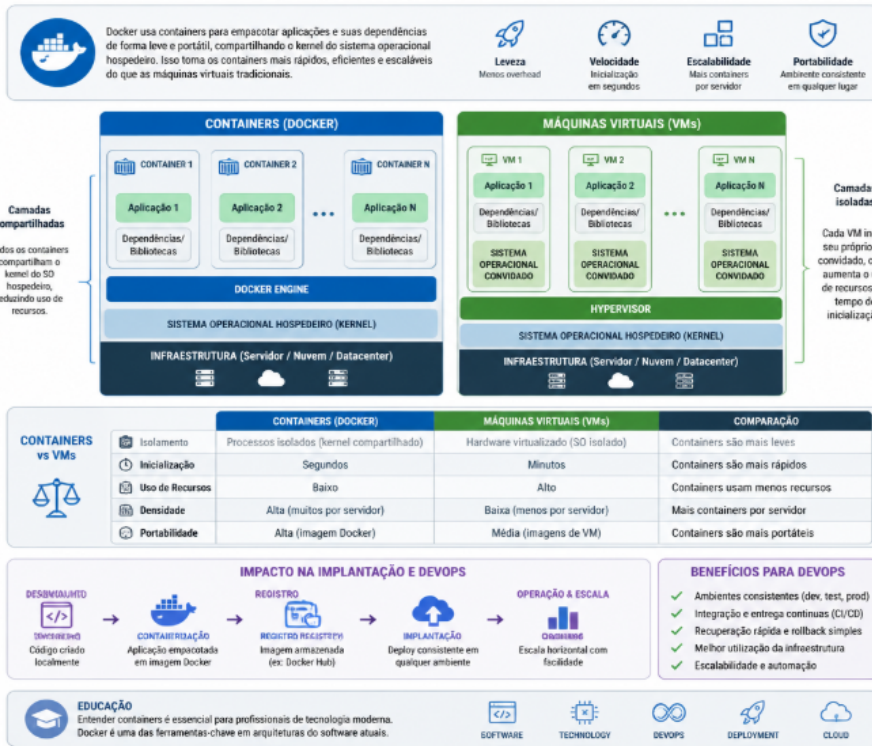


Gerência de Configurações e Automação

IaC para operações de TI modernas e escaláveis

VIRTUALIZAÇÃO COM CONTAINERS (DOCKER)

Conceito: Containers vs Virtual Machines



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



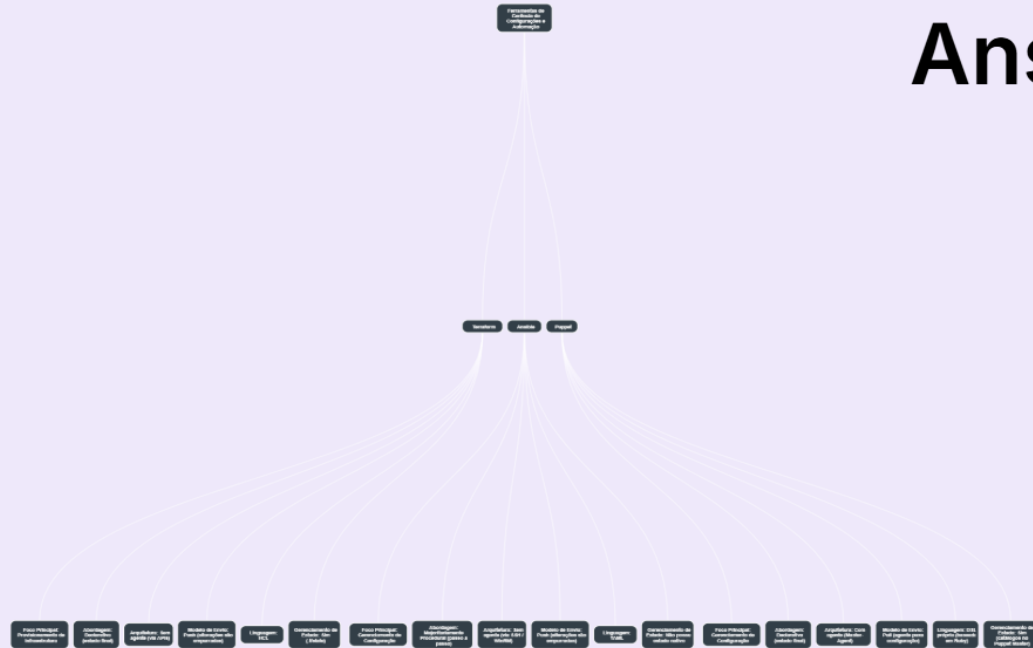
MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Comparando Terraform, Ansible e Puppet



Terraform, Ansible e Puppet são ferramentas essenciais em Gerência de Configurações e Automação (que é o processo de automatizar a forma como os sistemas são configurados e mantidos). Embora tenham objetivos comuns, eles se diferenciam em foco principal, arquitetura e abordagem. Compreender essas distinções é crucial para escolher a ferramenta certa para cada necessidade de automação de infraestrutura.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Sumário

- Revolução da Automação DevOps e IaC
- Terraform: Provisionamento de Infraestrutura
- Ansible: Gerência de Configuração Ágil
- Puppet: Conformidade Contínua e Escala
- Interoperabilidade e Ciclo de Vida



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Revolução da Automação DevOps

A **Gerência de Configurações** e a *Infraestrutura como Código* (IaC) representam uma mudança de paradigma na administração de sistemas. Em vez de intervenções manuais em servidores individuais, tratamos a infraestrutura como software: versionável, testável e repetível. As ferramentas líderes — **Terraform, Ansible e Puppet** — formam o alicerce para escalar operações de TI modernas, permitindo que equipes gerenciem milhares de servidores com a mesma precisão de uma única máquina. O objetivo central é eliminar o erro humano, garantir a conformidade e acelerar o 'time-to-market'.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Automação Imperativa

Neste modelo, o administrador define a **sequência exata de comandos** que devem ser executados para configurar um sistema. É como dar um roteiro passo a passo: "instale o pacote X, configure o serviço Y, inicie o Z". Embora direto, este método pode dificultar a gestão de estados complexos e a garantia de que as configurações persistam em ambientes diferentes ou após intervenções manuais, gerando inconsistências.

Automação Declarativa

Aqui, o foco está no **estado final desejado** do sistema, não nos passos para alcançá-lo. O administrador declara: "quero 3 instâncias com 4GB de RAM". A ferramenta de IaC (como Puppet ou Ansible) calcula e executa as ações necessárias para atingir e manter esse estado, garantindo a *idempotência* (aplicar a configuração múltiplas vezes sem alterar o estado final, se já estiver correto). Prioriza-se este modelo na IaC moderna para facilitar documentação e GitOps.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Terraform: O Provisionador

O **Terraform**, desenvolvido pela HashiCorp, é o padrão de mercado para provisionamento de infraestrutura em nuvem (AWS, Azure, GCP). Seu foco é criar e destruir os recursos base através de uma linguagem altamente legível chamada HCL (*HashiCorp Configuration Language*). Como um orquestrador, ele gerencia o ciclo de vida completo de diversos recursos, incluindo redes e VPCs, instâncias de computação e bases de dados gerenciadas, definindo a infraestrutura de forma declarativa para replicação consistente.

Provedor
(AWS,
Azure,
GCP)

Rede/VPC

Sub-rede

Instância de
Computação
(EC2)

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Como o Terraform Funciona

- Os **Arquivos de Configuração (.tf)** definem o estado desejado da infraestrutura utilizando a HashiCorp Configuration Language (HCL).
- O **State File (terraform.tfstate)**, um arquivo JSON crucial, atua como a 'única fonte da verdade', mapeando a configuração local para os recursos reais provisionados na nuvem.
- O **Plano de Execução** é gerado com ``terraform plan``, permitindo uma visualização detalhada de todas as alterações propostas antes que sejam efetivamente aplicadas com ``terraform apply``, garantindo previsibilidade e segurança nas operações.



INSTITUIÇÃO EXECUTORA



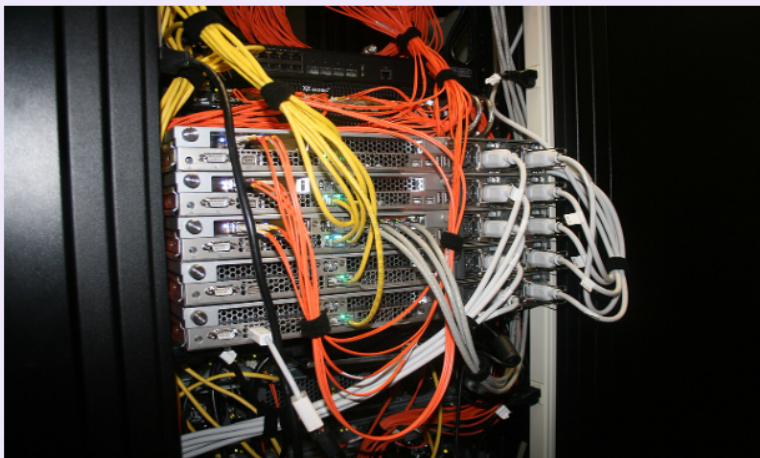
COORDENADORA



APOIO



Ansible: Configuração Ágil



Servidores NASA: gerenciamento de dados com automação e configuração ágil.

Prosseguindo da criação da infraestrutura com Terraform, o **Ansible** foca na configuração de software dentro das máquinas provisionadas. Ele se destaca pela sua simplicidade e agilidade, sendo *agentless* (não requer instalação de agente nos servidores de destino), utilizando protocolos como SSH para Linux ou WinRM para Windows para enviar comandos. É ideal para tarefas do "Dia 1", como instalação de pacotes, *deploy* de aplicações e configuração de serviços de segurança (Firewalls, usuários, patches). Sua configuração é feita através de arquivos YAML, conhecidos como Playbooks, o que facilita a curva de aprendizado para administradores de sistemas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Arquitetura do Ansible

Inventário de Hosts

O **Inventário** é uma lista centralizada que especifica os servidores ou *hosts* que o Ansible irá gerenciar. Ele pode ser **estático**, definido em um arquivo simples com endereços IP ou nomes de domínio, ou **dinâmico**, gerado sob demanda por *scripts* que consultam APIs de provedores de nuvem (como a AWS, Azure ou GCP), sistemas de CMDB (Configuration Management Database) ou outras fontes, permitindo flexibilidade e escalabilidade para gerenciar infraestruturas que mudam frequentemente.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Arquitetura do Ansible

Playbooks para Automação

Os **Playbooks** são o coração da automação no Ansible, escritos em formato YAML. Eles descrevem sequências de tarefas a serem executadas nos *hosts* definidos no inventário. Cada *playbook* pode conter um ou mais *plays*, onde cada *play* direciona um conjunto de *hosts* e define as tarefas a serem realizadas. Um exemplo de tarefa pode ser a instalação de um pacote, como ``apt install nginx``, a configuração de um serviço ou a cópia de arquivos. Eles são idempotentes, garantindo que a execução repetida atinja o estado desejado sem efeitos colaterais indesejados.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Arquitetura do Ansible

Módulos do Ansible

Os **Módulos** são pequenos programas, geralmente escritos em Python, que o Ansible executa nos *hosts* remotos. Eles são as ferramentas que realizam as ações específicas definidas nos *playbooks*. O Ansible vem com uma vasta biblioteca de milhares de módulos prontos para uso, cobrindo uma ampla gama de funcionalidades, desde gerenciamento de pacotes (`apt`, `yum`), serviços (`systemd`), usuários, arquivos, até a interação com bancos de dados, dispositivos de rede (como Cisco, Juniper) e APIs de nuvem. Esta modularidade permite que o Ansible seja extremamente versátil e extensível.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Puppet: Conformidade Contínua



Após abordarmos o Ansible para configurações ágeis, focaremos no **Puppet**, uma ferramenta fundamental para a manutenção de longo prazo e **conformidade** (compliance) em ambientes corporativos rígidos. Ele opera no modelo *agente-servidor*: um *Puppet Agent*, instalado na máquina de destino, consulta periodicamente o *Puppet Master* centralizado. O objetivo central é prevenir o *Configuration Drift* (desvio de configuração), garantindo que os sistemas mantenham o estado desejado ao longo do tempo. O ciclo de conformidade do Puppet envolve: 1. O agente verifica o estado atual da configuração. 2. Ele detecta qualquer mudança manual não autorizada ou desvio do estado definido. 3. O agente força o sistema a retornar ao estado ideal especificado pelo Puppet Master.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Manifestos e Catálogos

1

Definir Configuração

No Puppet, as configurações desejadas para a infraestrutura são definidas em arquivos chamados **Manifestos** (.pp). Estes são escritos em uma *linguagem de domínio específico* (DSL) que é baseada em Ruby, permitindo expressar o estado final desejado para os recursos do sistema.

2

Enviar Fatos (Facts)

O Agente Puppet, executando em cada nó cliente, coleta informações detalhadas sobre o próprio sistema, conhecidas como 'Facts' (fatos). Estes dados, que incluem sistema operacional, endereço IP, memória, etc., são então enviados para o Puppet Master, fornecendo o contexto atual do nó.

3

Compilar Catálogo

Com base nos Manifestos e nos 'Facts' recebidos do Agente, o Puppet Master compila um **Catálogo**. Este catálogo é um documento que descreve o estado final desejado para *aquele nó específico*, incluindo a lista de recursos que devem ser gerenciados e suas dependências, adaptado às suas características.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Manifestos e Catálogos

4

Aplicar e Reportar

O Catálogo compilado é enviado de volta ao Agente. O Agente, então, aplica localmente as configurações descritas no Catálogo, realizando as alterações necessárias para atingir o estado desejado. Após a aplicação, o Agente reporta o sucesso (ou falha) da operação de volta ao Master, garantindo a rastreabilidade e conformidade.

5

Escalabilidade do Modelo

Este modelo de operação, com a separação clara entre a definição de configuração (Manifestos), o estado atual do sistema (Facts) e a aplicação da política (Catálogo), confere ao Puppet uma alta escalabilidade. Ele é particularmente eficiente para gerenciar ambientes complexos com milhares de servidores, sejam eles físicos ou *bare-metal*.



INSTITUIÇÃO EXECUTORA



COORDENADORA



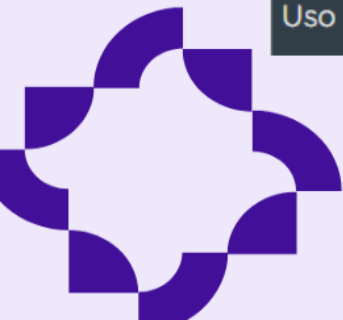
APOIO



Comparativo de Ferramentas

Este quadro compara as principais características do Terraform, Ansible e Puppet, destacando seus focos, arquiteturas e casos de uso ideais.

Carac.	Terraform	Ansible	Puppet
Foco Principal	Provisionamento (Orquestração)	Gerência de Configuração	Gerência de Configuração
Arquitetura	Agentless (API Cloud)	Agentless (SSH/WinRM)	Agent-based (Pull)
Linguagem	HCL (Declarativa)	YAML (Declarativa/Sequencial)	Puppet DSL (Declarativa)
Gestão de Estado	State File local/remoto	Não possui estado intrínseco	Master-based (Catálogo)
Uso Ideal	Criar VPCs, EC2, RDS	Deploy de App, Patching	Compliance em larga escala



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



O Ciclo de Vida da Automação

1

Provisionamento (Dia 0)

Nesta fase inicial, o **Terraform** é empregado para estabelecer a infraestrutura fundamental. Ele provisiona recursos essenciais na nuvem, como redes virtuais, instâncias de máquinas (servidores), sistemas de armazenamento e políticas de segurança, criando a base sobre a qual todos os outros componentes serão construídos.

2

Configuração Inicial (Dia 1)

Imediatamente após o provisionamento da infraestrutura, o **Ansible** entra em ação. Seu papel é configurar os servidores recém-criados, instalando softwares e dependências, ajustando as configurações do sistema operacional e realizando o *deploy* inicial de aplicações. Esta etapa garante que os sistemas estejam prontos para operação.

3

Manutenção e Conformidade (Dia 2)

A fase de manutenção se estende por toda a vida útil da infraestrutura. O **Puppet** é ideal para esta etapa, monitorando e aplicando continuamente a conformidade do estado dos servidores. Alternativamente, o Ansible pode ser usado via agendamentos (`cron`) ou plataformas como AWX/Tower para garantir que os sistemas permaneçam seguros e configurados corretamente, prevenindo desvios de configuração ao longo do tempo.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Estratégias de Interoperabilidade



Provisionamento com Callback

Esta abordagem integra o Terraform com o Ansible. Uma vez que o Terraform conclui o provisionamento de uma máquina virtual (VM) ou outro recurso de infraestrutura, ele executa automaticamente um *callback* (geralmente via ``local-exec`` no Terraform). Esse *callback* aciona um Playbook do Ansible, que então assume a tarefa de configurar o sistema operacional, instalar softwares e aplicar configurações específicas na VM recém-criada, garantindo uma transição fluida do provisionamento para a configuração.



Infraestrutura Imutável (Golden Images)

Com a *Infraestrutura Imutável*, o foco é em *Golden Images*. Primeiramente, ferramentas como o Packer são utilizadas para criar imagens de máquina (VMs ou contêineres) pré-configuradas. O Ansible desempenha um papel crucial aqui, configurando a imagem base com todas as dependências, softwares e configurações necessárias *antes* de ela ser utilizada. Em seguida, o Terraform é usado para distribuir e orquestrar essas instâncias imutáveis na nuvem. Se uma atualização for necessária, uma nova *Golden Image* é criada e as instâncias existentes são substituídas, em vez de serem modificadas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Estratégias de Interoperabilidade



Sinergia Operacional

A combinação estratégica dessas ferramentas resulta em uma **redução drástica** do tempo de deploy. Processos que tradicionalmente levariam horas são concluídos em questão de minutos. Essa sinergia elimina os *silos* operacionais entre as equipes de provisionamento de infraestrutura (Terraform) e as equipes de configuração de sistemas (Ansible). A automação ponta a ponta melhora a consistência, a repetibilidade e a agilidade, facilitando a entrega contínua de aplicações e serviços.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Um engenheiro de infraestrutura precisa garantir que, em um ambiente de produção crítico, qualquer alteração manual indesejada em um arquivo de configuração essencial seja detectada e revertida automaticamente, restabelecendo o estado de conformidade previamente definido. Qual ferramenta de Gerência de Configuração é a mais adequada para implementar essa capacidade de *remediação contínua*?

- A Terraform, pois ele gerencia o ciclo de vida da infraestrutura e detecta desvios automaticamente.
- B Packer, que é utilizado para criar imagens de máquinas virtuais imutáveis e padronizadas.
- C Ansible, devido à sua capacidade de execução remota de scripts e auditoria de configurações.
- D Puppet, por sua arquitetura baseada em agentes que monitora e corrige o *drift de configuração* de forma autônoma.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Um engenheiro de infraestrutura precisa garantir que, em um ambiente de produção crítico, qualquer alteração manual indesejada em um arquivo de configuração essencial seja detectada e revertida automaticamente, restabelecendo o estado de conformidade previamente definido. Qual ferramenta de Gerência de Configuração é a mais adequada para implementar essa capacidade de *remediação contínua*?

- ✗ Terraform, pois ele gerencia o ciclo de vida da infraestrutura e detecta desvios automaticamente.
- ✗ Packer, que é utilizado para criar imagens de máquinas virtuais imutáveis e padronizadas.
- ✗ Ansible, devido à sua capacidade de execução remota de scripts e auditoria de configurações.
- ✓ Puppet, por sua arquitetura baseada em agentes que monitora e corrige o *drift de configuração* de forma autônoma.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Verdadeiro ou falso?

O Terraform utiliza um arquivo de estado (state file) para mapear a infraestrutura real à configuração desejada.

← Falso

Verdadeiro →

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





✓ Verdadeiro

O Terraform utiliza um arquivo de estado (state file) para mapear a infraestrutura real à configuração desejada.

Explicação

É fundamental para o Terraform manter um arquivo de estado que representa a infraestrutura real e, assim, calcular o delta de mudanças a serem aplicadas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Verdadeiro ou falso?

No ciclo de vida DevOps, o Terraform é geralmente associado às tarefas de 'Dia 0', como a criação de redes e instâncias de nuvem.

← Falso

Verdadeiro →

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





✓ Verdadeiro

No ciclo de vida DevOps, o Terraform é geralmente associado às tarefas de 'Dia 0', como a criação de redes e instâncias de nuvem.

Explicação

O Terraform é predominantemente usado para o provisionamento inicial da infraestrutura (Dia 0), estabelecendo os alicerces como redes e máquinas virtuais antes da implantação de aplicações.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Verdadeiro ou falso?

O Ansible requer a instalação de um agente de software em todos os nós gerenciados antes de poder executar qualquer tarefa.

a ← Falso

Verdadeiro →

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





× Falso

O Ansible requer a instalação de um agente de software em todos os nós gerenciados antes de poder executar qualquer tarefa.

Explicação

O Ansible é uma ferramenta *agentless*, o que significa que ele não requer a instalação de agentes nos nós gerenciados, utilizando SSH (para Linux) ou WinRM (para Windows) para comunicação.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Discuta com o colega ao lado

Considerando os cenários de uma startup e de um banco tradicional, qual combinação de Puppet, Ansible e Terraform seria mais adequada para cada um e por quê?

Foque nas implicações arquiteturais (Agentless vs. Agent-based) e nas necessidades específicas de agilidade, segurança e infraestrutura legada.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Conclusão

- Terraform se destaca no provisionamento declarativo de infraestrutura em nuvem usando HCL e gerencia o estado.
- Ansible oferece configuração ágil e agentless via SSH/WinRM, ideal para instalação de pacotes e deploy de apps.
- Puppet assegura conformidade contínua e reversão de 'Configuration Drift' através de seu modelo agente-servidor.
- A sinergia entre Terraform, Ansible e Puppet otimiza o ciclo de vida DevOps, de provisão à manutenção.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

